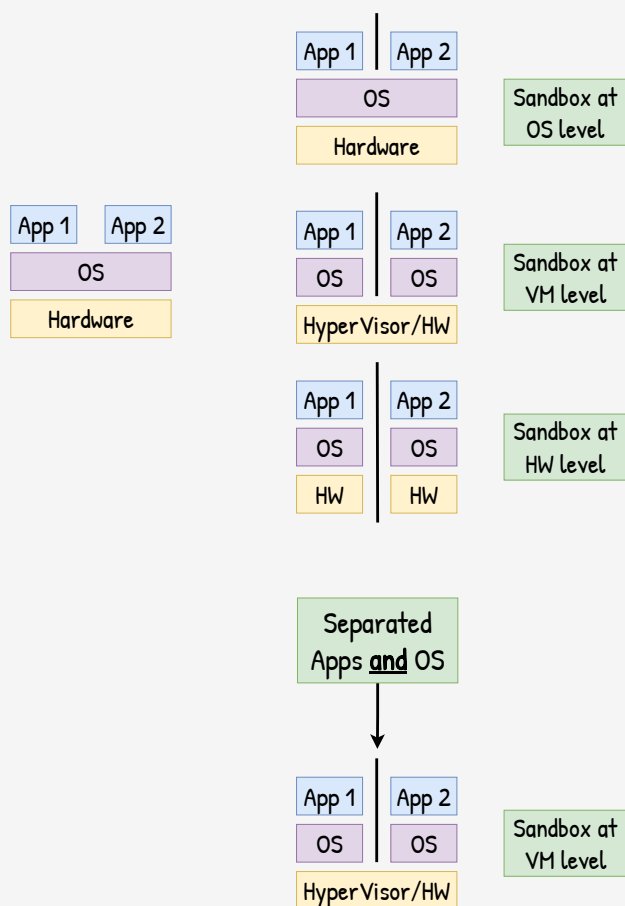


Sandbox at VM level

Virtual Machine, Virtualization, Emulation, Hypervisor, Row Hammering

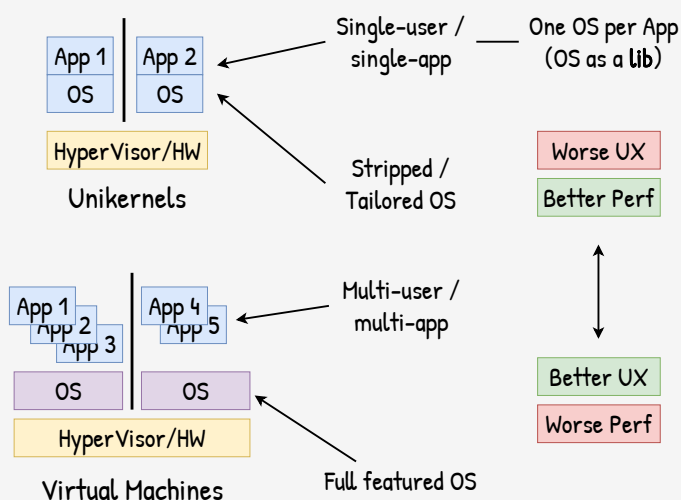
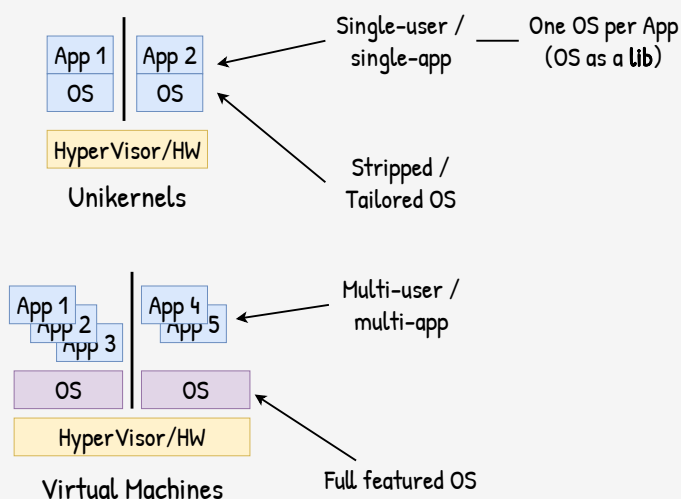
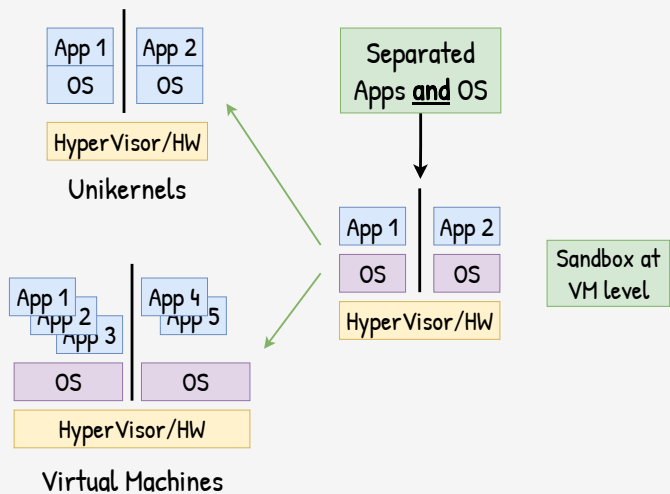
Taller de Seguridad

Ing. Martin Di Paola - Fiuba



Como los vimos en la clase pasada: queremos hacer que las apps corran aisladas las unas de las otras. Podemos enforzar dicha separación a nivel del OS, a nivel de una VM (virtual machine) o incluso a nivel físico (separación del hardware)

Cuando vamos a nivel de una VM, cada virtual machine tiene su propio sistema operativo y este, no habla directamente con el hardware sino con el hypervisor (en breve veremos esto).



Hay 2 grandes familias: las virtual machine propiamente dichas y los unikernels.

Las VMs tienen un OS completo con soporte multi-usuario y multi-app (como cualquier OS moderno).

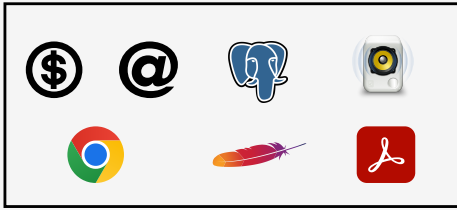
Los unikernels son un OS modificado (muchas veces a medida de que la app se está sandboxeando). Son en general OS con soporte para un único user y app.

Uno puede ver a los unikernels como si fueran una librería más de la app. Así, las syscalls se transforman en llamadas directas y no hay una separación real entre user y kernel space.

La magia de los unikernels es que tienen una mejor performance, en especial un menor consumo de memoria. La contra es que requieren de compiladores especiales, son un tech experimental y, al tener todas las apps separadas, tienen una peor UX (más difícil de usar).

Los unikernels pueden tener un caso de uso en servidores que tengan una sola app (por ejemplo, un nginx) donde la UX es un issue menor y se prioriza seguridad y memoria.

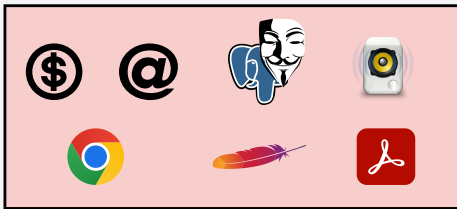
Las VMs en cambio requieren un setup mínimo, tienen una buena UX (las apps dentro de una VM se comunican sin problema). Todo a costa de mayor consumo de memoria.



VM

Mixed
Concerns

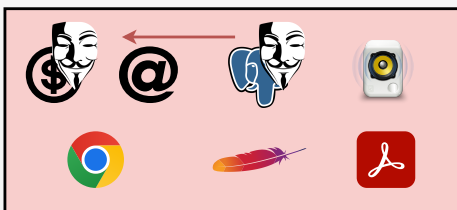
Con VM donde pueden correr múltiples apps, que apps deberían estar?



VM

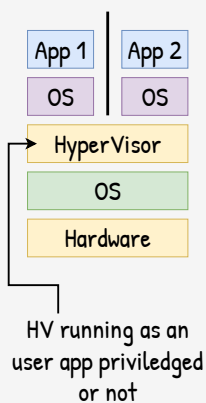
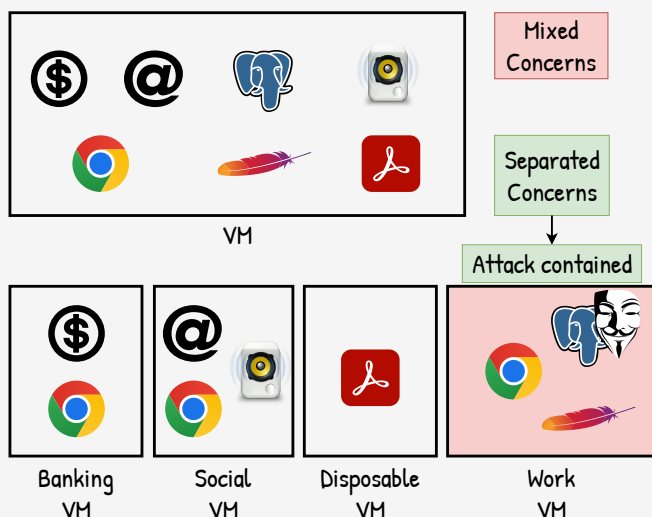
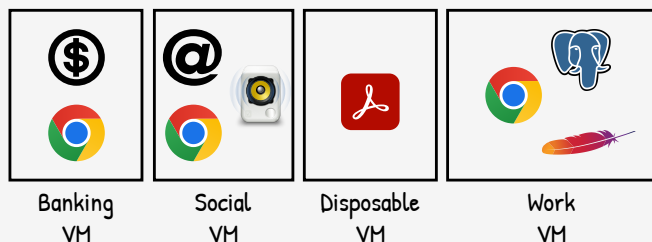
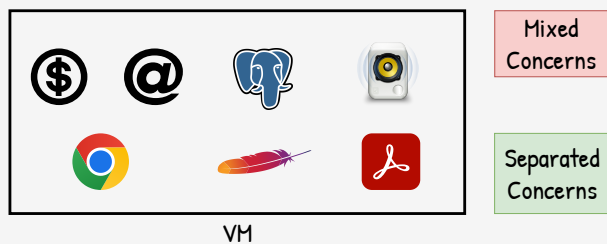
Mixed
Concerns

Si ponemos todas las apps, una sola que quede pawneada y toda la VM va a quedar pawneada.



VM

Mixed
Concerns



La estrategia es separar los concerns en “dominios” de seguridad considerando daño/impacto, intención y probabilidad de pawneo.

Si piensas que te pawneen una red social o que te pawneen varias tiene el mismo daño/impacto para vos, y que te pawneen una es tan probable como q te pawneen otra (digamos que es por que todas se acceden con el browser y especulas q el atacante ira por un bug del browser), entonces pueden ir todas en una VM.

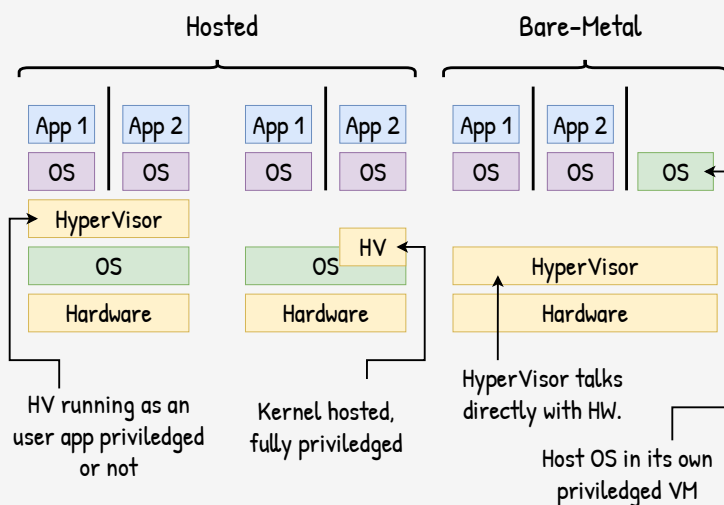
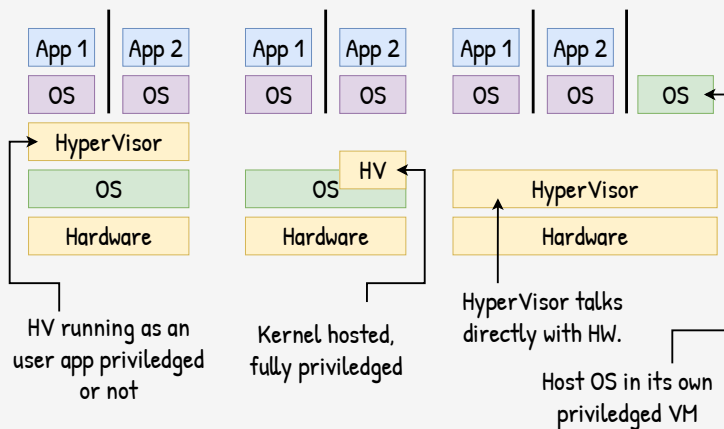
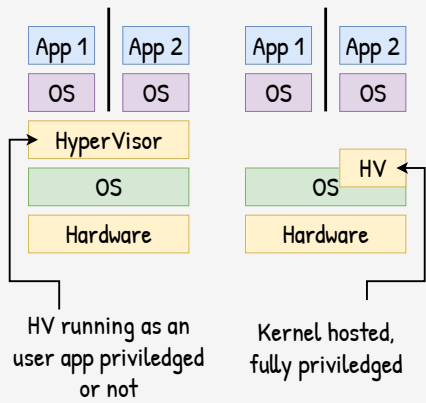
Las apps de un lab de tu trabajo (server web, DBs) conectadas a la red del trabajo (VPN) podrian ser seguras hasta que un atacante quiera pawnearte para poder entrar a la red del trabajo (vos no sos el target real).

Asi, la intención te diferencia este atacante de otro que quiera pawnearte el homebanking.

Ahora veamos q es un hypervisor (HV). El primer tipo es una app que corre con los mismos privilegios que una app de user y sobre un OS.

En esta categoría caen los emuladores (más en breve)

Los emuladores tienen una pésima performance. Por que no hacer que el OS del host nos ayude a “emular” las VMs.

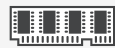


Y si sacamos al OS del host directamente? El hipervisor puede ser quien tenga acceso al hardware directo y los drivers (el OS del host) corran en una VM privilegiada.

Xen fue pionero en esta idea y hoy es el corazón de todo el cloud computing.

Microsoft, una década tarde, le esta siguiendo los pasos.

A los primeros se los llama *hosted* y a los segundos *bare-metal*.



Mem



CPU



Motherboard

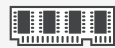


Net



Disk

OS



Mem



CPU

Boot

Basic setup, load
OS from disk/net
and kicks it in



Motherboard



Net



Disk

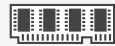
Net

Net Driver

FS

Disk Driver

OS



Mem



CPU

Boot

Then, OS takes
control of the
hardware



Motherboard



Net



Disk

Net

Net Driver

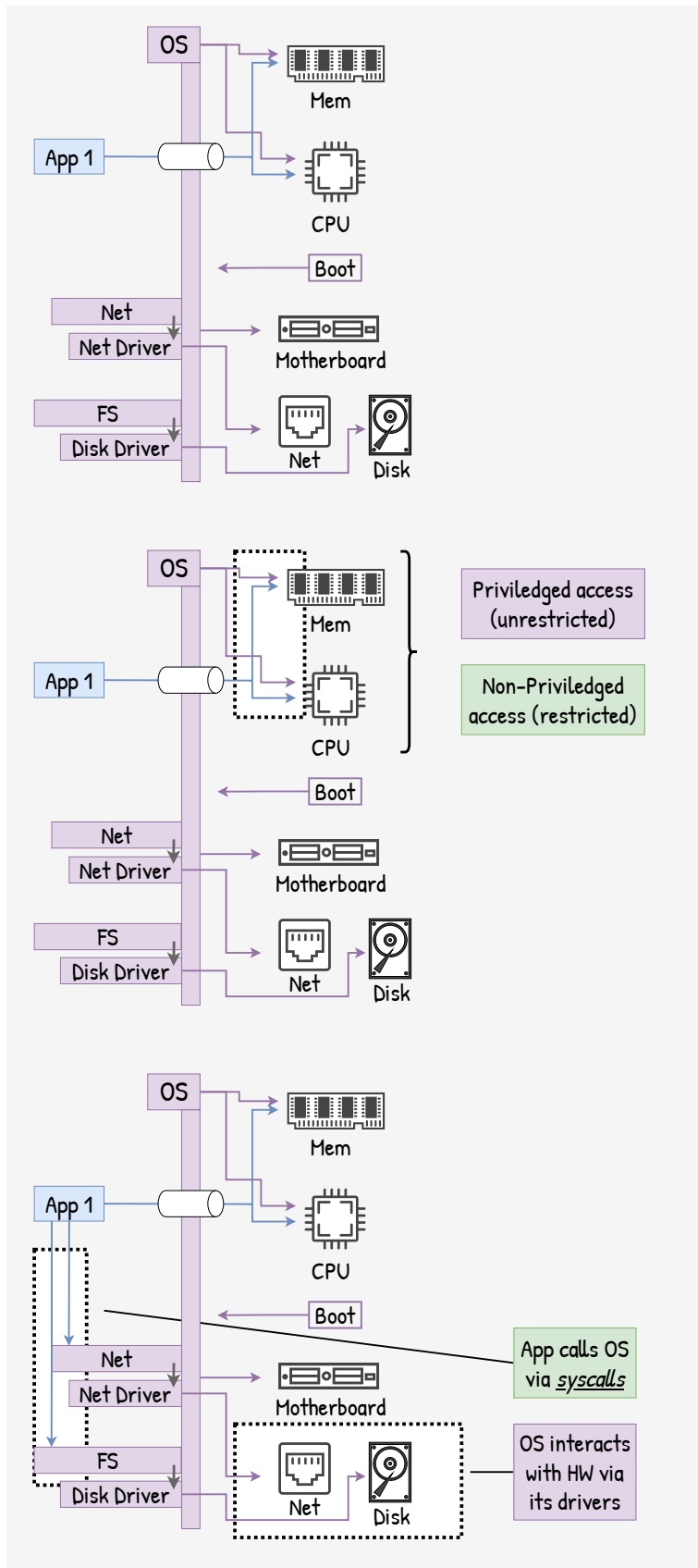
FS

Disk Driver

En que consiste el hardware de una máquina? CPU, memoria y periféricos y quien los conecta, la placa madre (motherboard)

El prenderse la maquina se inicia desde el hardware el proceso de booteo. Este inicializa lo mínimo, y ejecuta al OS.

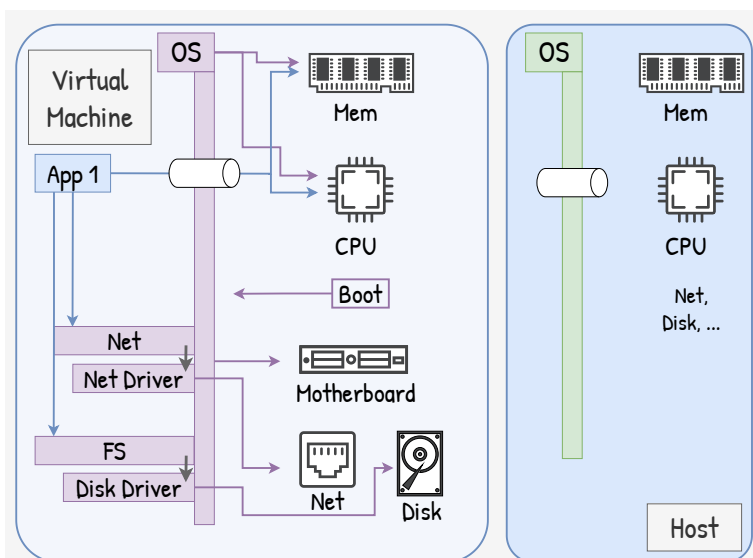
Con el OS cargado (o en proceso), es el OS quien termina de inicializar y tomar control de todo el hardware.



Ahora tanto el kernel como las apps pueden hacer uso del hardware. CPU y memoria son accedidos directamente por ambos (el kernel impone controles de acceso a las apps del user)

Pero si una app quiere acceder a otro hardware (lease disco, interfaz de red), esta accede indirectamente a travez de una abstracción del OS (el file system o el stack de network) via llamadas al sistema (syscall).

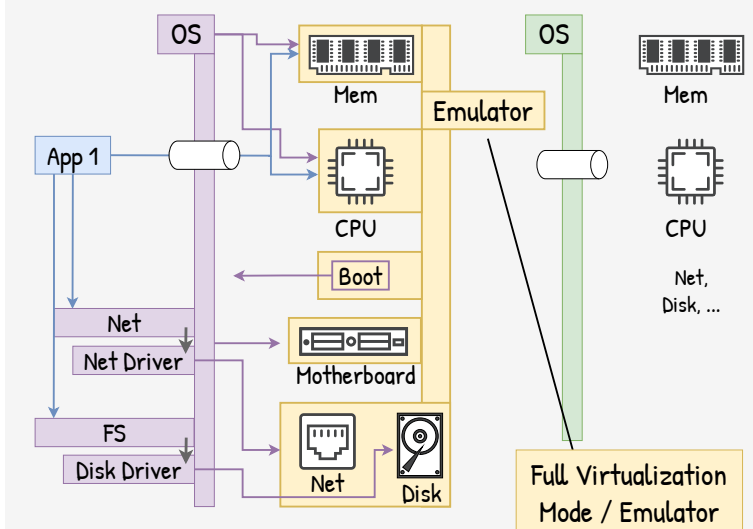
Son estas abstracciones del kernel quien delegan en los drivers el detalle fino de la interacción con el hardware.



Todo lo visto, incluido el hardware, ahora es parte de la virtual machine.

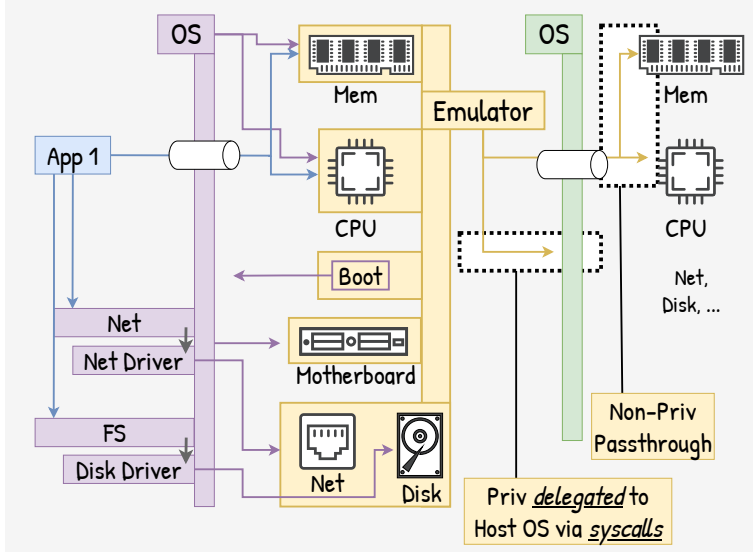
Detras de todo esta el verdadero OS con el verdadero hardware en el host.

Si viste la película The Matrix, ahora estas tomando la pastilla roja.

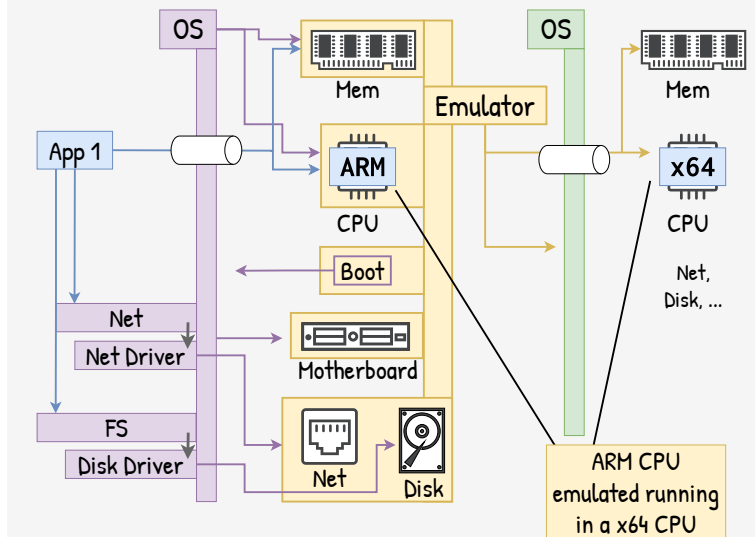
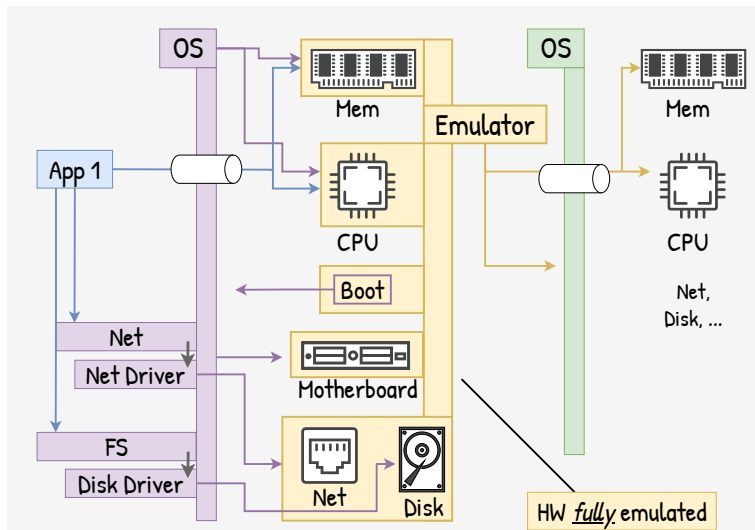


El primer approach es tener un app que emule todo el hardware: CPU, motherboard, discos, monitores, keyboard, y demas fierros.

Esto se lo conoce como Full Virtualization Mode.

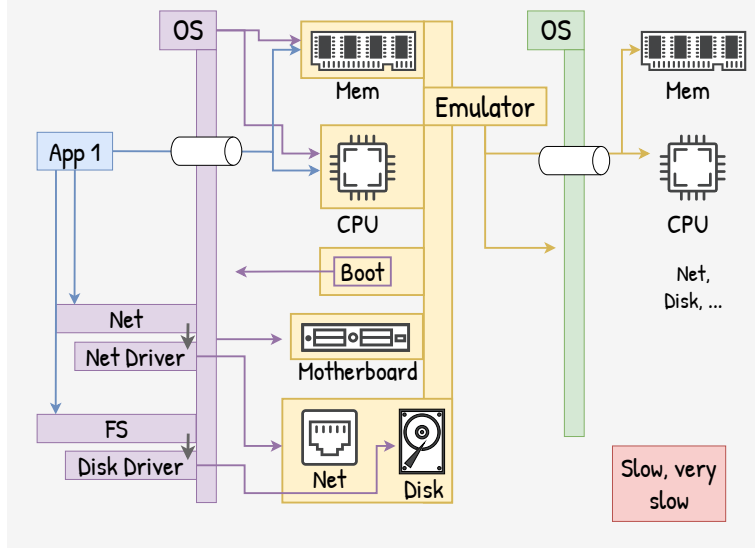


Como cualquier app no privilegiada, el emulador accede a la CPU y memoria del host directamente pero al resto del hardware via syscalls.

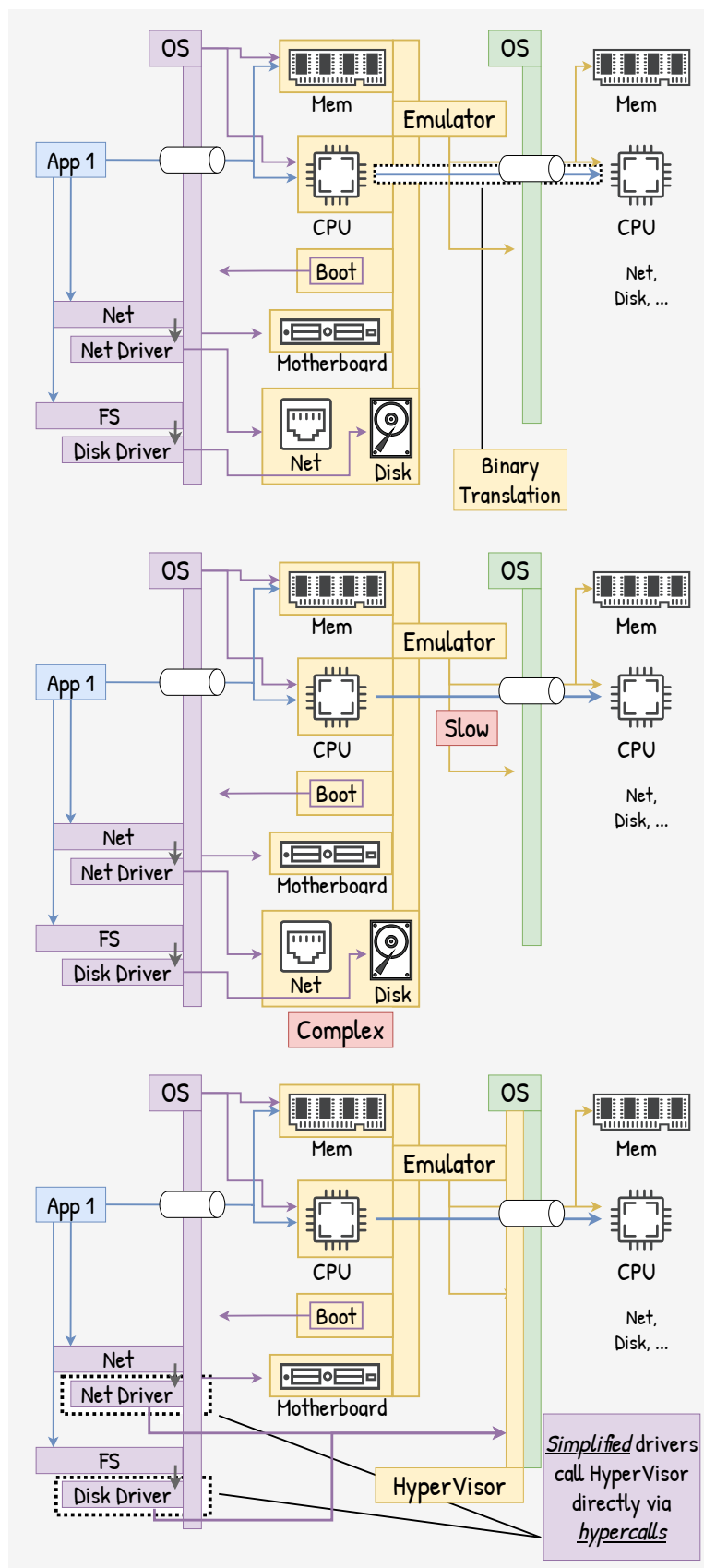


La CPU también esta emulada por lo que podriamos correr código en una arquitectura en un host con una CPU de otra (ARM sobre AMD).

Muy útil si queres probar una app de celular en tu notebook!



Pero si, es lento. Muy lento.



Una optimización es hacer binary translation. Al igual que Java o Python toman código y los compilan a código nativo en real-time (just in time o JIT), un emulador toma código para la arquitectura de la VM y lo traduce a la del host.

La primer vez es mas lento pero el resto de las veces el emulador ejecuta directamente código nativo sin tener que emular instrucción a instrucción.

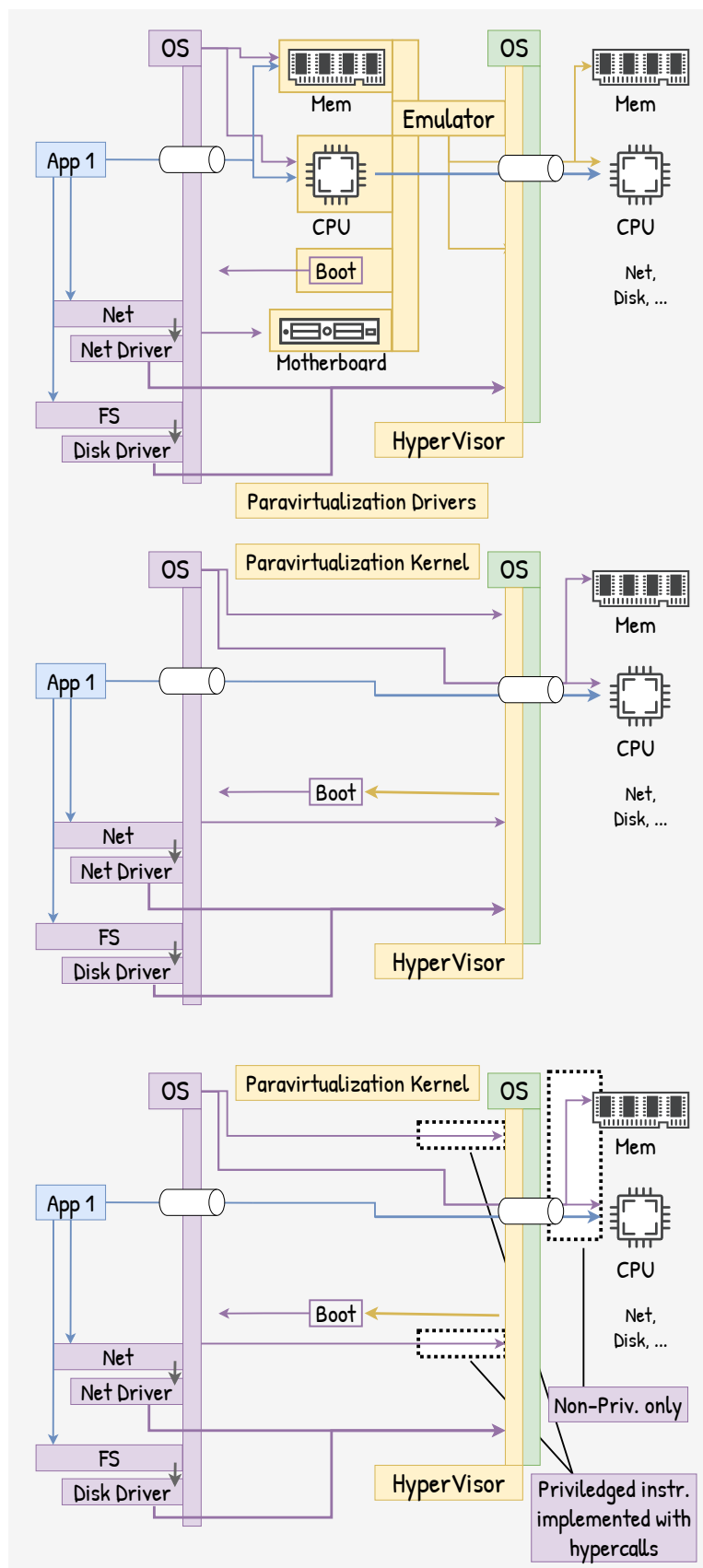
Pero aun asi, es lento. Y un emulador esta lejos de ser simple.

Si la app en la VM logra triggerear un bug en el emulador, esta va a correr fuera del sandbox.

Y con la complejidad de los emuladores, no es una cuestión de si tienen bugs sino de si fueron descubiertos o no.

Y si en vez de emular el hardware el hypervisor le ofreciera una API al OS de la VM para interactuar con el real?

El OS de la VM tiene mucho menos código: sus drivers serían llamadas al hypervisor (hypercalls) y nada más. No habría emulación (menos código y más rapidez) y la seguridad no se vería comprometida (el OS de la VM no tiene acceso directo al hardware sino que es el hypervisor, a travez del OS del host)



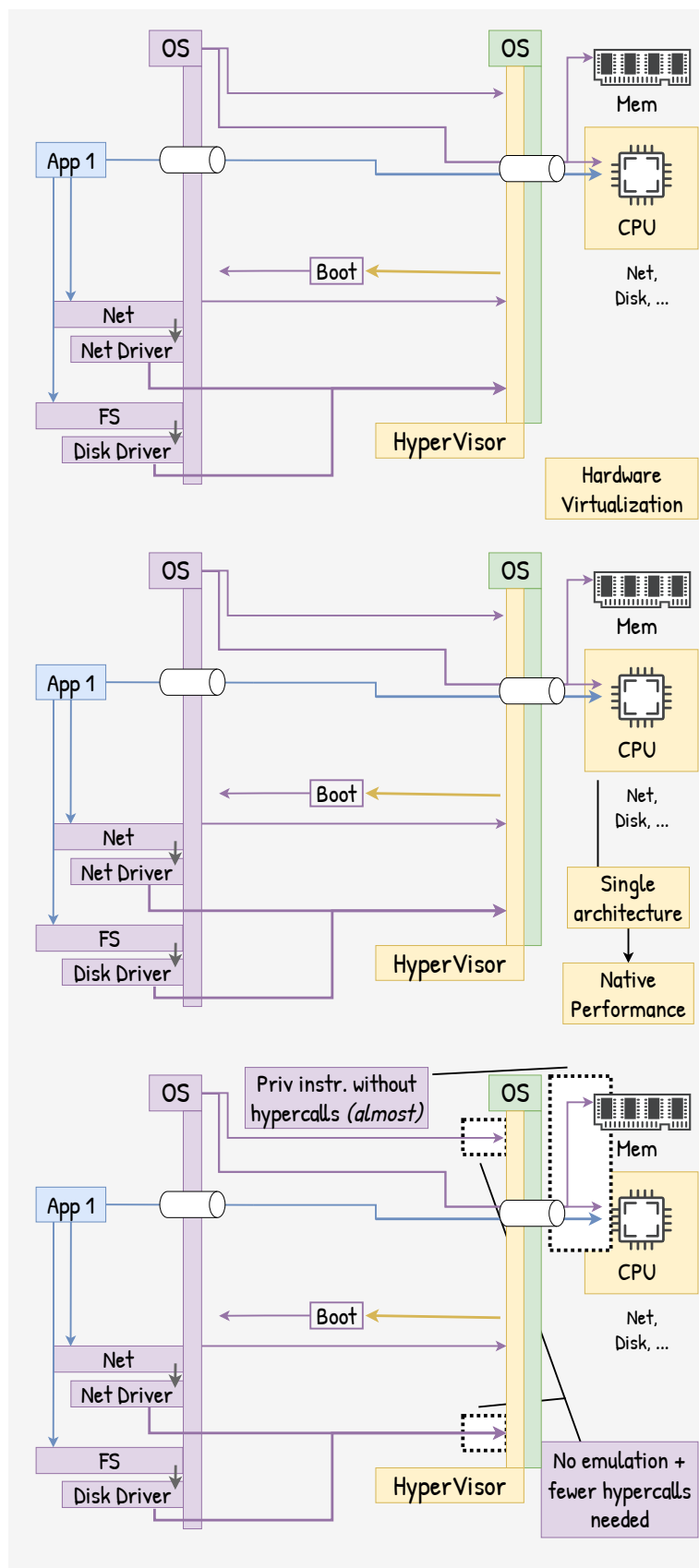
Y un detalle no menor. Realmente no hay que modificar al OS de la VM, solo un cambio de drivers.

Esto se lo conoce como paravirtualización de drivers.

Si además realizamos modificaciones en el OS, un OS que este al tanto que esta corriendo dentro de una VM puede no intentar interactuar con un hardware (q no existe y habria q emular) y delegar en el hypervisor (via hypercalls) todo lo necesario.

En especial, control de CPU y memoria: el OS de la VM debe saber que realmente no tiene privilegios reales por estar en una VM y debe permiso al OS del host.

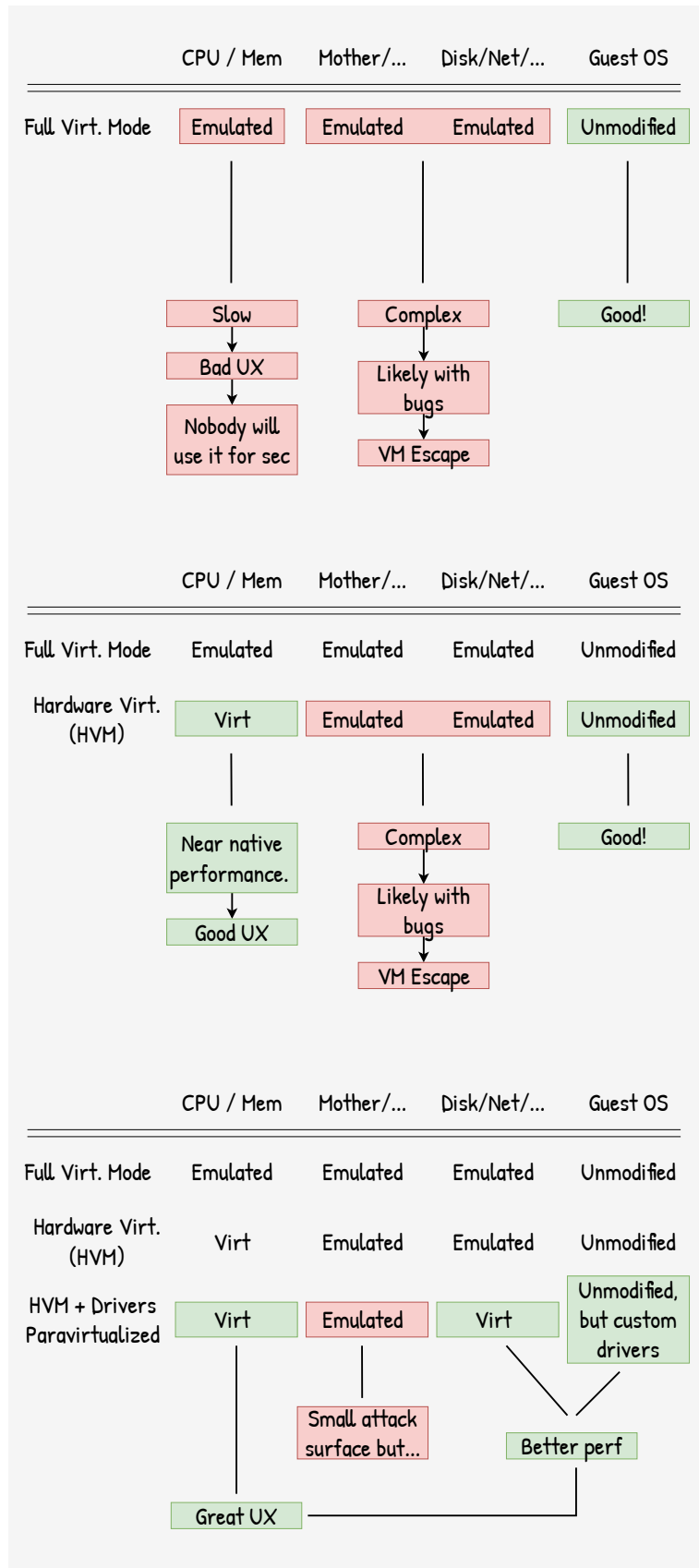
Esto se lo conoce como paravirtualización del kernel.



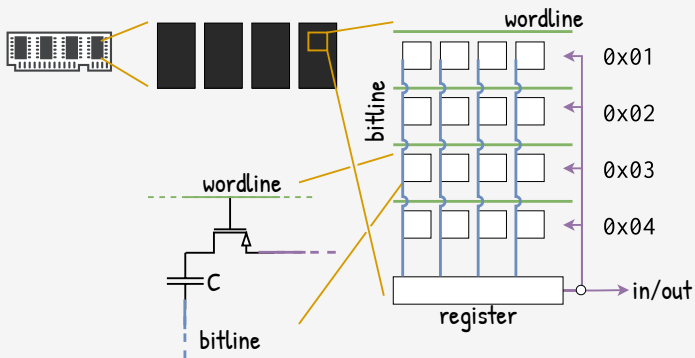
Así como el kernel puede estar al tanto de estar en una VM y ser más eficiente, el hardware puede estar al tanto de que va a correr VMs y ofrecer aislamiento y optimizaciones desde los fierros.

Esto se lo conoce como hardware virtualization (HVM) aunque cada empresa de hardware tiene nombres específicos como Intel VT-x (intel), AMD-V (amd).

Con ayuda del hardware, no hay necesidad de emular la CPU y en particular el OS de la VM puede hacer uso de ciertas operaciones privilegiadas (no todas) con menor interacción del hypervisor haciendo que la VM corra muchísimo más rápido.

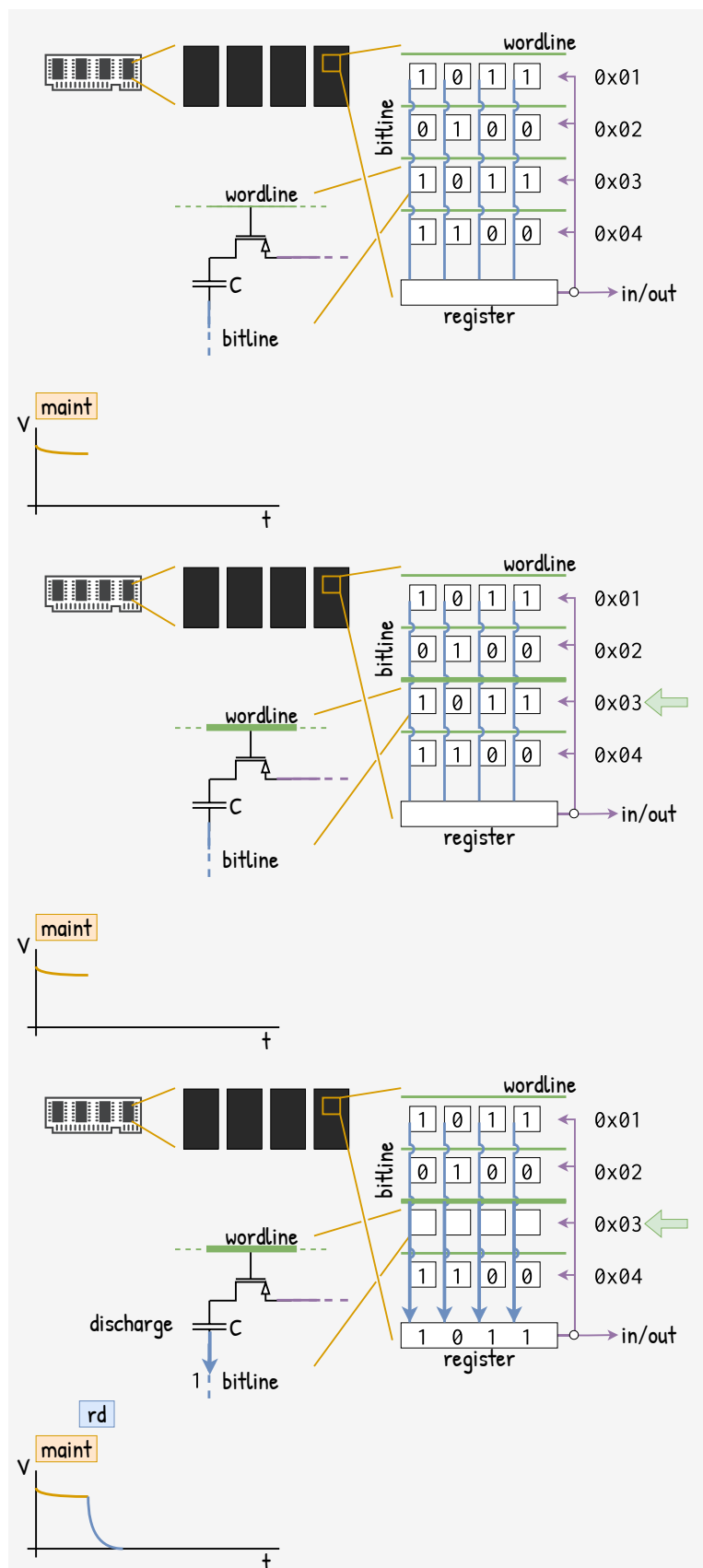


	CPU / Mem	Mother/...	Disk/Net/...	Guest OS
Full Virt. Mode	Emulated	Emulated	Emulated	Unmodified
Hardware Virt. (HVM)	Virt	Emulated	Emulated	Unmodified
HVM + Drivers Paravirtualized	Virt	Emulated	Virt	Unmodified, but custom drivers
HVM + Kernel PV (PVHVM / PVH)	Virt	Virt	Virt	Modified
	Native performance	Minimal exposition		Require guest OS support



Los chip de memoria RAM almacenan los bits (0 o 1) en capacitores (segun esten o no cargados).

Cada conjunto de capacitor + transistor se organizan en filas (palabras de memoria) controladas por la misma wordline y estas filas se organizan en matrices para formar un banco de memoria.

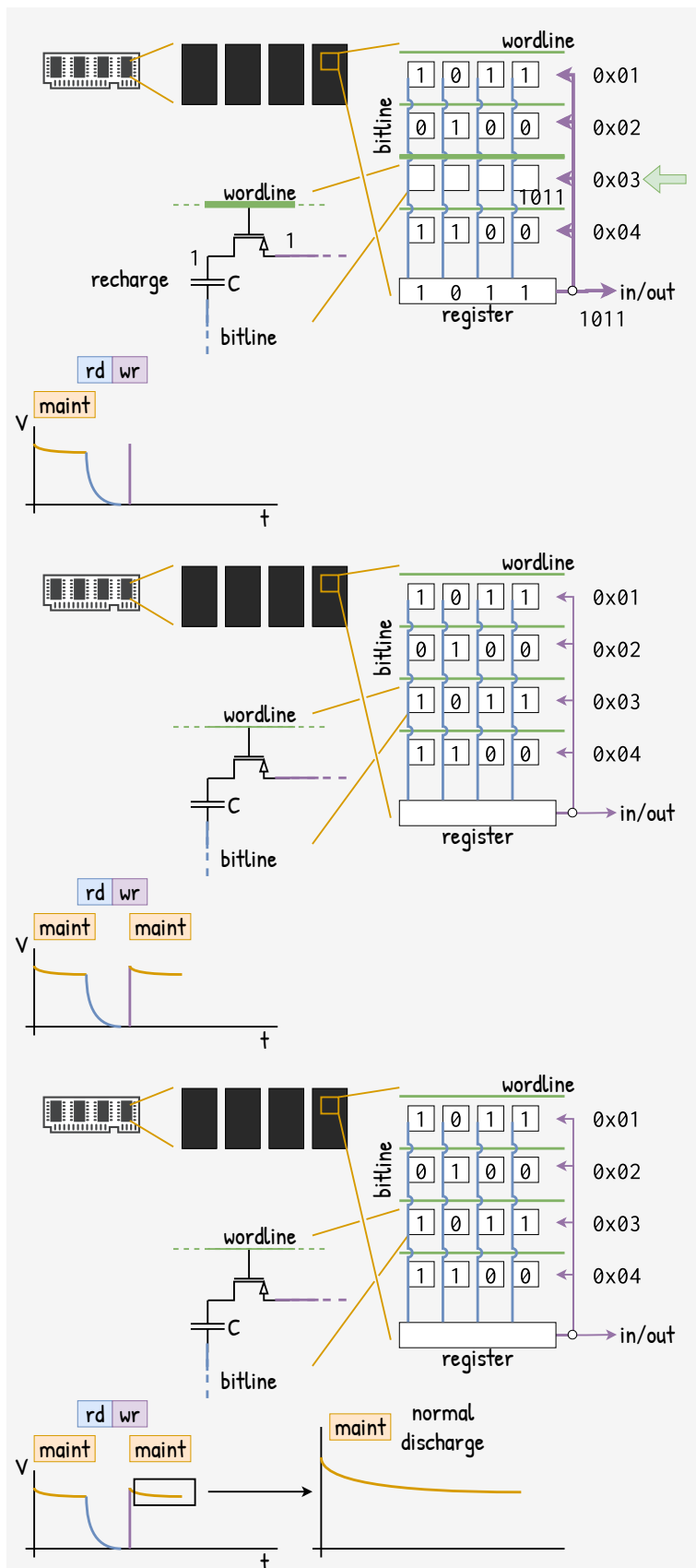


Cuando un capacitor esta cargado, este va perdiendo la carga lentamente debido a interacciones con el resto del circuito aunque esta descarga es mínima.

Cuando se quiere hacer una lectura, la wordline correspondiente a la palabra se activa (energiza), esto habilita al transistor y el capacitor se descarga enviando la señal por la bitline hasta un registro.

El registro es una circuiteria especial para contener información temporal durante la lectura.

Notar como el proceso de lectura destruye la información dado q los capacitores se descargan.



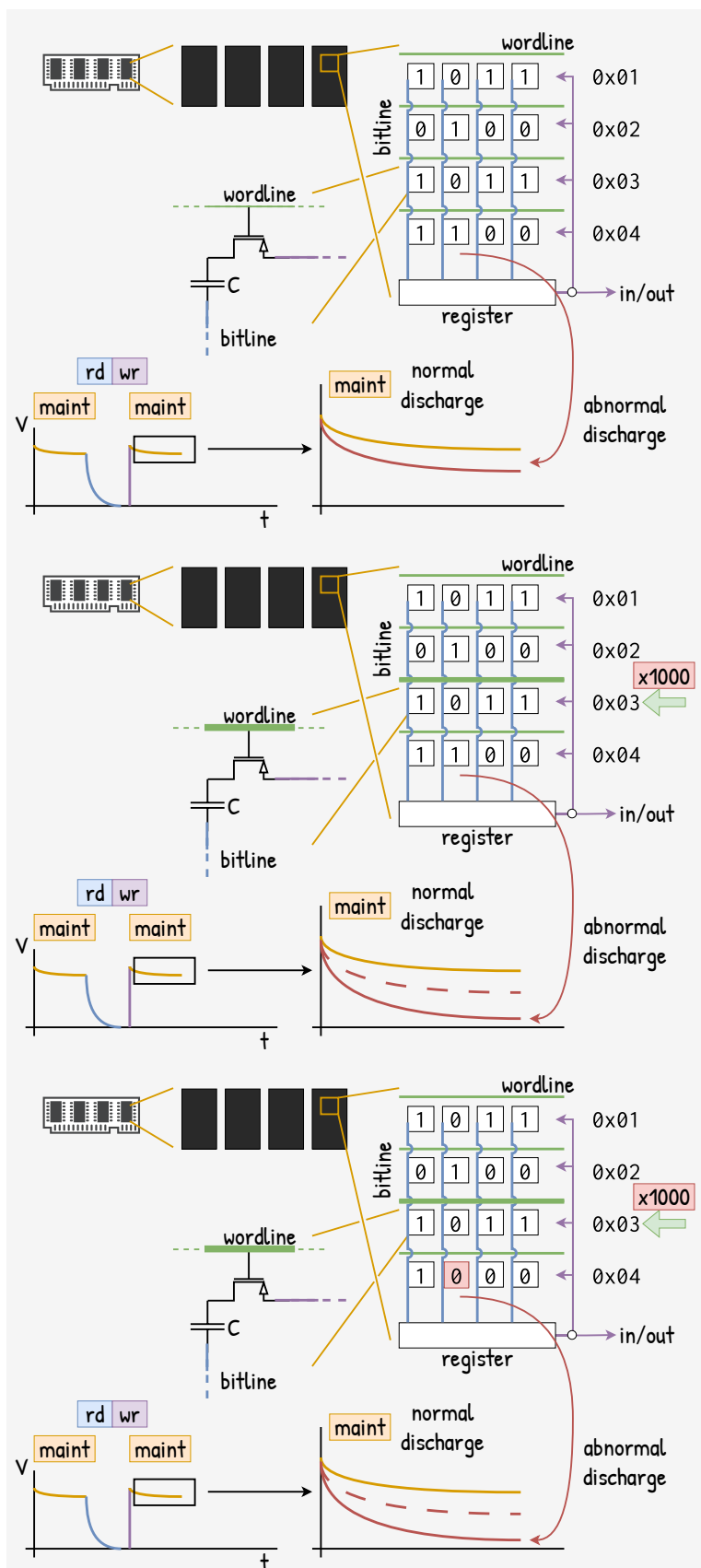
Como parte del proceso de lectura, los bits leídos y guardados en el registro son copiados y enviados al exterior (CPU) pero también son enviados nuevamente a los capacitores para volver a cargarlos.

Una operación de lectura termina haciendo también una operación de escritura.

Tras completada la operación de lectura, la carga de los capacitores se reestablece.

Es normal que los capacitores se descarguen de forma natural por leaks y pérdidas. Es algo predecible y la circuitería interna de la memoria RAM hace una recarga periódica para asegurarse que la información no se pierde.

Como se hace la recarga? Solo tiene que hacer lecturas!

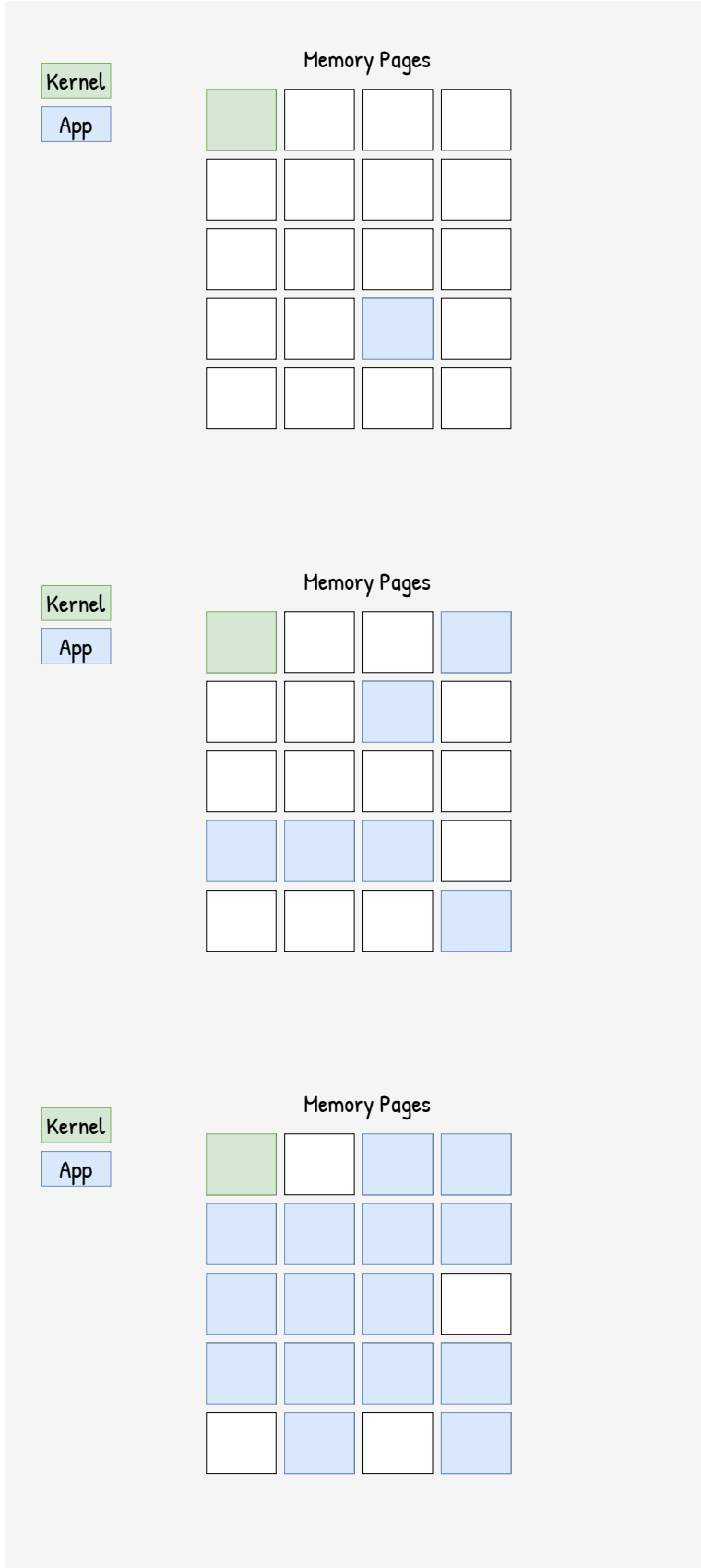


Pero debido a la lectura (y escritura) de una fila, los capacitores aledaños físicamente pueden tener una descarga algo más rápida.

Si una misma dirección de memoria (léase, una misma fila de capacitores) se lee miles de veces, esto produce una descarga mucho más rápida en capacitores de filas aledañas.

Y con algo de suerte, algún capacitor aledaño habrá perdido la suficiente cantidad de carga que la circuitería interpretara como un bit 0 en vez de 1.

Leer miles de veces una misma dirección de memoria produjo un bitflip (escritura) en otra!

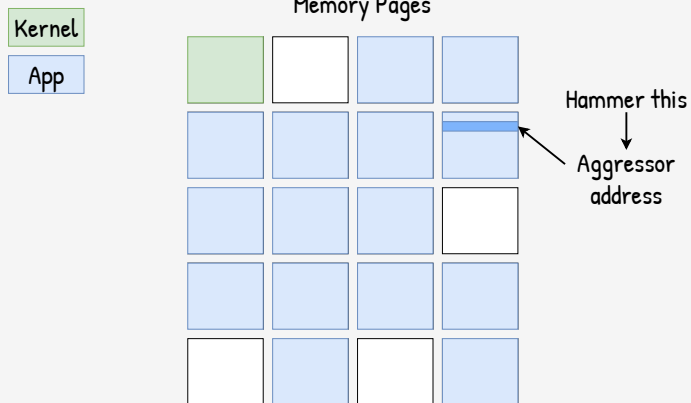


Pensaras que este glich del hardware es improbable y mucho menos útil.

Are you sure?

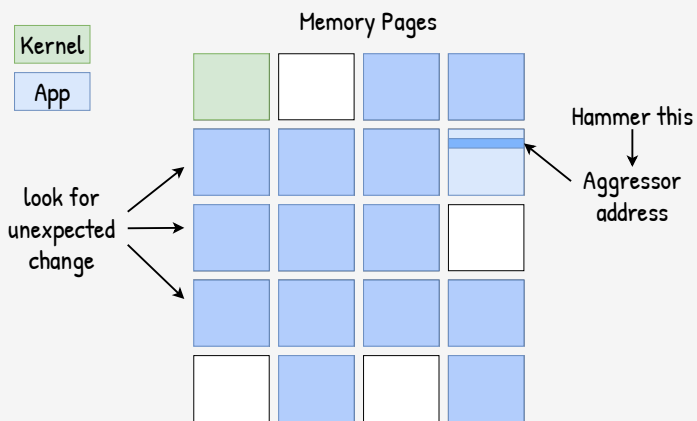
Imaginate las páginas de memoria de un sistema. Algunas estan asignadas a una app del user y otras al kernel (y otras sin asignar).

En la primer fase de un ataque, la app empieza a allocar tantas páginas como pueda.

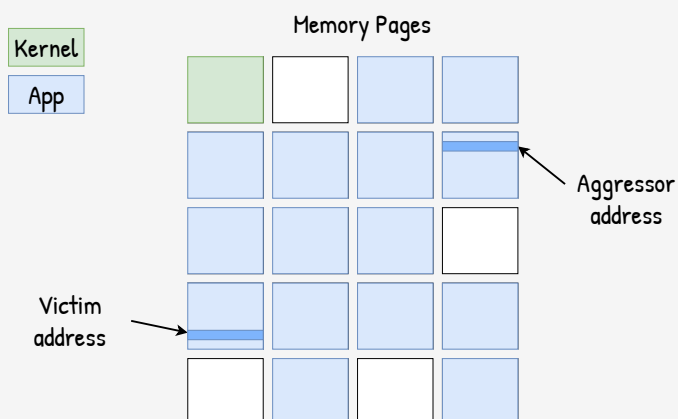


Luego, toma una dirección y hace miles de lecturas, martillando a la fila de capacitores.

El atacante no sabe a cual, solo sabe que con algo de suerte habra un bitflip en alguna lado.



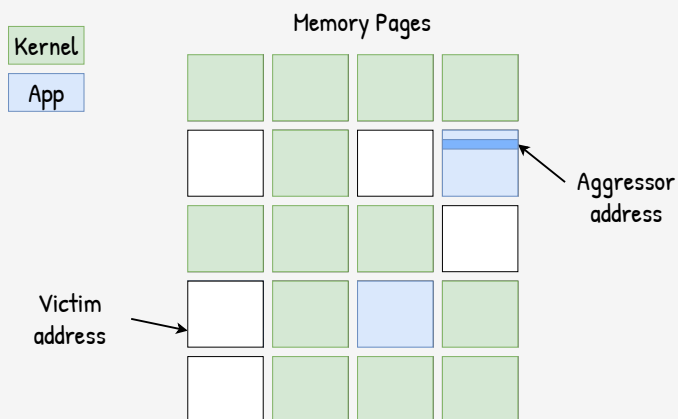
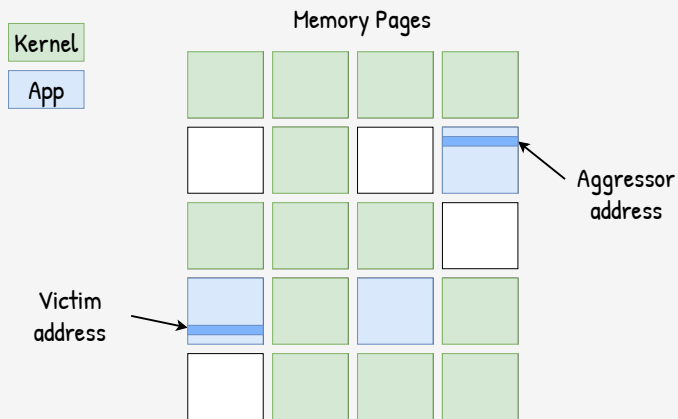
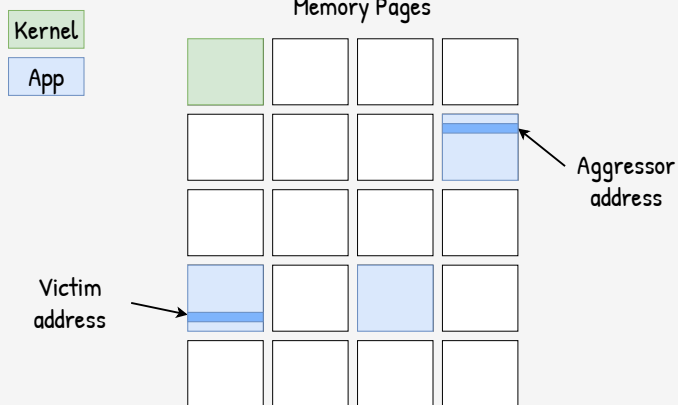
Luego del martilleo, el atacante busca en sus paginas si alguna de ellas tuvo un cambio, un bit flip.



Puede que el hammering no haya sido efectivo y no encuentre ninguna row cambiada. Puede q haya sido efectivo pero el cambio hizo bit flips de bits q el atacante no le interesan.

Si es asi, repite el proceso martillando a otra dirección.

Con algo de tiempo lograra encontrar un par de direcciones que le sirvan: una para hacer el hamemring (la fila agresora) y la otra q sufre del bitflip (la file víctima)



Podras decir, pero cual es el punto? Acaso la página donde esta la dirección víctima no la controlaba ya el atacante?

Si, pero esto es solo la primer parte del ataque.

Con el par agresor-victima, la app libera todas las páginas salvo aquellas 2 q contienen esas 2 direcciones.

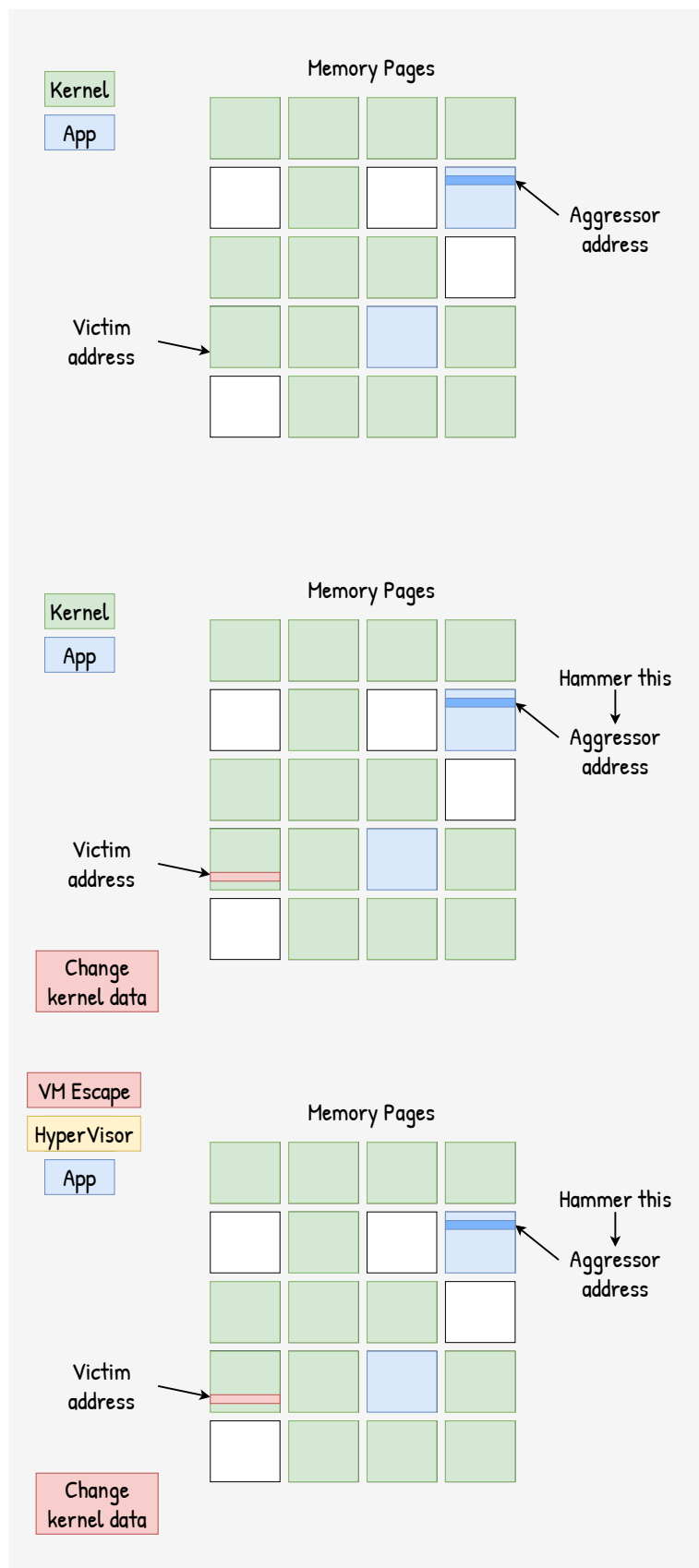
Ahora, la app induce al kernel a allocar páginas.

Por ejemplo cuando una app le pide al kernel mapear un archivo a memoria, el archivo queda mapeado en una paginal de la app *pero* el kernel tiene que trackear que paginas y que permisos tiene cada mapping.

Esta son las entries de la Page Table.

Si la app mapea archivos minúsculos muchas veces, el kernel va a tener q trackear miles de esos mappings. Y ese tracking el kernel lo guarda en memoria, requiriendo que el kernel también alloque paginas para él.

Con casi todas las paginas del sistema allocadas, la app libera la pagina q contiene la dirección víctima.



Y ahora, la app sigue forzando al kernel a allocar más memoria para él.

Con una buena probabilidad el kernel va a terminar allocando para el la página q contiene a la víctima.

Más aun, con buena probabilidad el atacante sabe que estructuras de datos el kernel va a escribir ahi. Va a escribir las entries de la Page Table.

Y ahora, la estocada. La app martillea con lecturas la dirección agresora para tener un bitflip en la dirección víctima.

Y con esto estaremos haciendo una escritura en memoria del kernel!

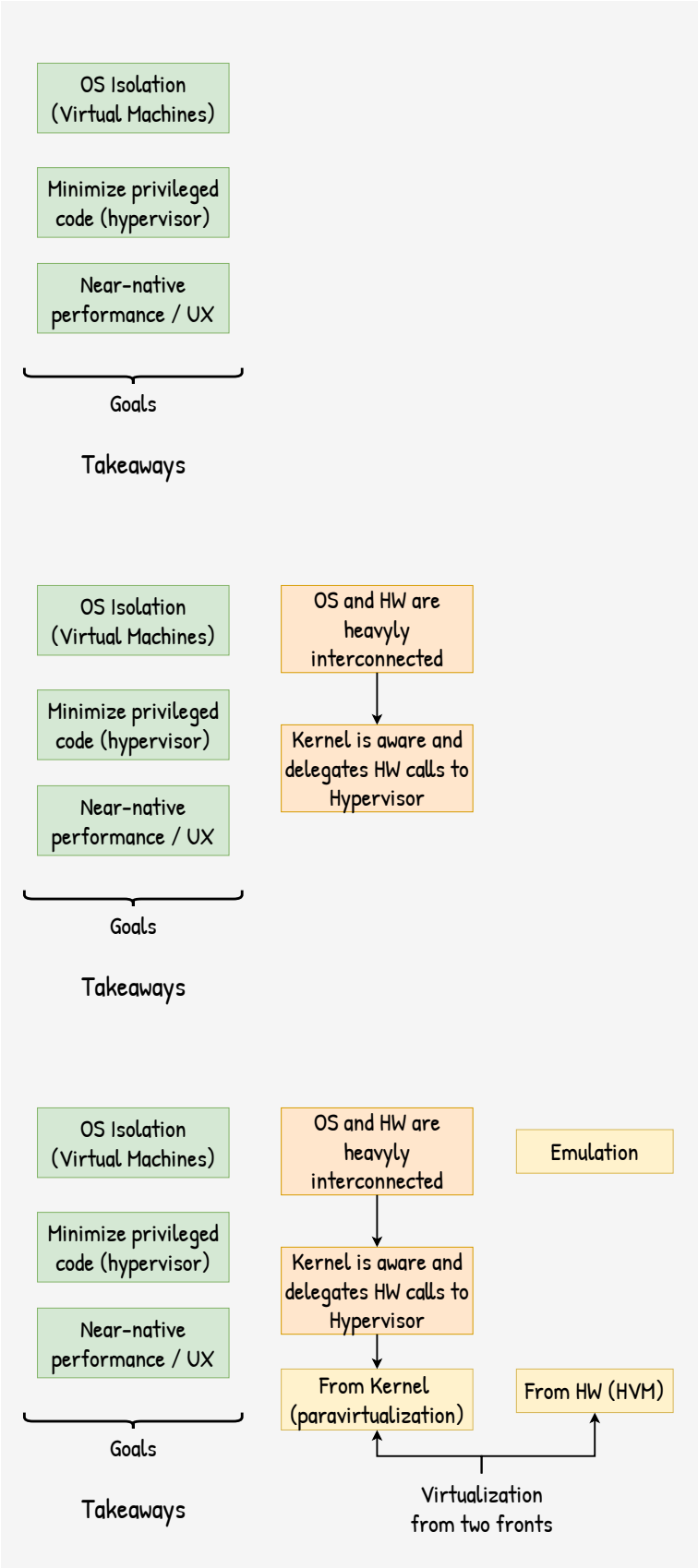
Más aun, si elegimos bien a la víctima (buscamos una que flippee un bit en específico), podemos cambiar el bit que controla los permisos de las páginas.

El kernel no dara acceso a páginas que no deberíamos!

Por ejemplo si la app le pidió al kernel mapear como read-only un programa setuid (como **ping**), con el bitflip ahora ese mapping quedaría como read-writeable.

La app ahora puede modificar un programa setuid para que en vez de hacer, digamos un **ping**, habra un shell como root!

Y si el atacante es creativo puede transformar un bit flip no solo en un priv escalation (a root) sino manipular la memoria del hypervisor y escapar de la VM.

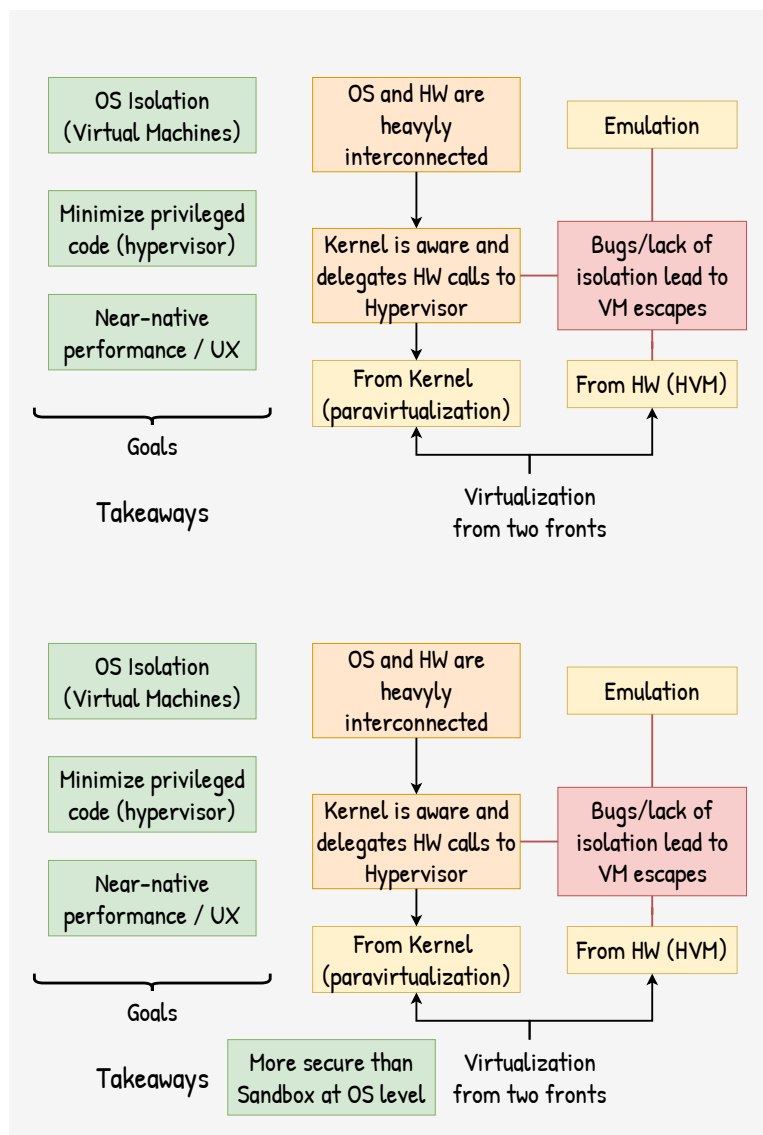


El sandboxing con VMs requiere q sea difícil para una app escaparse del sandbox y que la UX sea casi la misma que correr la app fuera de una VM.

Alcanzar esto tiene tradeoffs.

Depende de con qué tecnología se cuente uno puede emular, paravirtualizar o virtualizar el hardware.

Siempre uno debe elegir aquello que simplifique lo que el hypervisor tiene que hacer (más simple -> menos bugs).



Bugs habrá siempre, sea en el software como en el hardware.

Las VMs son mejores sandboxes que los implementados por el OS pero no estan libres de vulnerabilidades.

Dependiendo del daño/impacto, probabilidad e intención de un ataque uno debe elegir q tecnología es lo mas adecuada.

Un threat model correcto es vital para no armar un setup inseguro o complejo innecesariamente.